

JK&R Support Portal - React Source Code

```
import React, { useState, useEffect, useRef } from 'react';
import {
  Shield, MessageSquare, HelpCircle, Trophy, User, LogOut,
  Send, Lock, ChevronRight, CheckCircle, Clock, EyeOff, Search,
  Users, MessageCircle, Bot, ShieldAlert
} from 'lucide-react';

// --- GEMINI API SETUP ---
const apiKey = ""; // API key is injected by the execution environment

// --- CONSTANTS ---
const MASTER_KEY = "SPARTAN2026"; // The ultimate key for admin access

const ROLES = {
  ADMIN: ['Manager', 'Marketing Manager', 'Canvas Manager', 'Other (HQ)'],
  USER: ['Setter', 'Closer', 'Other (Field)']
};

// Custom Spartan Helmet SVG component
const SpartanHelmet = ({ className }) => (
  <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5"
  strokeLinecap="round" strokeLinejoin="round" className={className}>
    <path d="M12 2C8.5 2 6 4.5 6 9v3c0 2.5 1.5 4 3 5l1 3h4l1-3c1.5-1 3-2.5
  3-5V9c0-4.5-2.5-7-6-7z"/>
    <path d="M12 2v7"/>
    <path d="M9 10h6"/>
    <path d="M12 12v3"/>
    <path d="M8 12c1.5 1 3 1.5 4 1.5s2.5-.5 4-1.5"/>
  </svg>
);

export default function App() {
  const [userProfile, setUserProfile] = useState(null);
  const [view, setView] = useState('support'); // 'support', 'chat', 'admin-chat',
  'leaderboard', 'profile', 'directory', 'dm', 'ai'
  const [activeDmUser, setActiveDmUser] = useState(null);

  // Data States (In-Memory for Claude compatibility)
  const [users, setUsers] = useState([]);
  const [tickets, setTickets] = useState([]);
  const [messages, setMessages] = useState([]);
  const [adminMessages, setAdminMessages] = useState([]);
  const [dms, setDms] = useState([]);

  // --- ACTIONS ---
  const handleLoginSignup = (formData) => {
    // Check if user already exists in our mock memory
    const existingUser = users.find(u => u.email.toLowerCase() ===
  formData.email.toLowerCase());

    if (existingUser) {
      setUserProfile(existingUser);
    } else {

```

```

const newProfile = {
  id: 'user-' + Date.now() + Math.random().toString(36).substr(2, 9),
  email: formData.email,
  name: formData.name,
  role: formData.role,
  isAdmin: formData.isAdminRole,
  score: 0,
  joinedAt: Date.now()
};

setUsers(prev => [...prev, newProfile]);
setUserProfile(newProfile);
}
};

const handleLogout = () => {
  setUserProfile(null);
  setView('support');
  setActiveDmUser(null);
};

const openDm = (targetUser) => {
  setActiveDmUser(targetUser);
  setView('dm');
};

if (!userProfile) {
  return <AuthScreen onComplete={handleLoginSignup} />;
}

return (
  <div className="flex h-screen bg-[#111317] text-gray-300 font-sans selection:bg-[#EAB308] selection:text-black">
    {/* SIDEBAR */}
    <div className="w-64 bg-[#15181C] border-r border-[#2A2E35] flex flex-col transition-all duration-300">
      <div className="p-6 border-b border-[#2A2E35]">
        <div className="flex items-center gap-3 mb-2">
          <SpartanHelmet className="w-9 h-9 text-[#EAB308]" />
          <div>
            <h1 className="text-[#EAB308] font-bold text-xs tracking-widest uppercase">JK&R Construction</h1>
            <h2 className="text-white font-semibold text-sm">Support Portal</h2>
          </div>
        </div>
      </div>
      <div>
        <nav className="flex-1 py-6 px-4 space-y-2 overflow-y-auto">
          <p className="text-[10px] text-gray-500 font-bold tracking-widest uppercase px-3 mb-2">Main</p>
          <NavItem icon={<HelpCircle size={18} />} label="Roadblocks & Q&A" isActive={view === 'support'} onClick={() => setView('support')} />
          <NavItem icon={<Users size={18} />} label="Team Directory" isActive={view === 'directory'} onClick={() => setView('directory')} />
          <NavItem icon={<Bot size={18} />} label="Spartan AI Coach" isActive={view === 'ai'} onClick={() => setView('ai')} />
        </nav>
        <div className="pt-4">

```

```

        <p className="text-[10px] text-gray-500 font-bold tracking-widest uppercase px-3
mb-2">Communications</p>
        <NavItem icon={<MessageSquare size={18} />} label="Public Chat" isActive={view
=== 'chat'} onClick={() => setView('chat')} />
        {activeDmUser && view === 'dm' && (
            <NavItem icon={<MessageCircle size={18} />} label={`Chat: $
{activeDmUser.name.split(' ')[0]}`} isActive={true} onClick={() => setView('dm')} />
        )}
        {userProfile.isAdmin && (
            <NavItem icon={<ShieldAlert size={18} />} label="HQ Admin Comms" isActive={view
=== 'admin-chat'} onClick={() => setView('admin-chat')} highlight={true} />
        )}
    </div>

    {userProfile.isAdmin && (
        <div className="pt-4">
            <p className="text-[10px] text-gray-500 font-bold tracking-widest uppercase
px-3 mb-2">Leadership</p>
            <NavItem icon={<Trophy size={18} />} label="Admin Leaderboard" isActive={view
=== 'leaderboard'} onClick={() => setView('leaderboard')} />
            </div>
        )}

    <div className="pt-4 pb-2">
        <p className="text-[10px] text-gray-500 font-bold tracking-widest uppercase px-3
mb-2">Personal</p>
        <NavItem icon={<User size={18} />} label="My Profile" isActive={view ===
'profile'} onClick={() => setView('profile')} />
    </div>
</nav>

<div className="p-4 border-t border-[#2A2E35] bg-[#1A1D23]">
    <div className="flex items-center gap-3 mb-4">
        <div className="w-10 h-10 rounded-full bg-[#2A2E35] flex items-center justify-
center text-[#EAB308] font-bold">
            {userProfile.name.charAt(0).toUpperCase()}
        </div>
        <div className="overflow-hidden">
            <p className="text-sm text-white font-medium truncate">{userProfile.name}</p>
            <p className="text-xs text-gray-500 truncate">{userProfile.role}</p>
        </div>
    </div>
    <button
        onClick={handleLogout}
        className="w-full flex items-center justify-center gap-2 text-xs text-gray-400
hover:text-white transition-colors py-2 rounded border border-[#2A2E35] hover:bg-[#2A2E35]"
    >
        <LogOut size={14} /> Log Out
    </button>
</div>
</div>

    {/* MAIN CONTENT AREA */}
    <div className="flex-1 flex flex-col overflow-hidden">
        <header className="h-16 flex items-center px-8 bg-[#111317] border-b border-[#2A2E35]
shrink-0">
            <h2 className="text-xl font-semibold text-white tracking-wide">
                {view === 'support' && 'Roadblocks & Q&A'}

```

```

        {view === 'directory' && 'Team Directory'}
        {view === 'chat' && 'Public Comms Channel'}
        {view === 'admin-chat' && 'HQ Admin Secure Comms'}
        {view === 'dm' && `Direct Message: ${activeDmUser?.name}`}
        {view === 'leaderboard' && 'Engagement Leaderboard'}
        {view === 'profile' && 'Profile Settings'}
        {view === 'ai' && 'Spartan AI Sales Coach'}
      </h2>
    </header>

    <main className="flex-1 overflow-y-auto p-8 relative">
      {view === 'support' && <SupportSystem userProfile={userProfile}
setUserProfile={setUserProfile} tickets={tickets} setTickets={setTickets}
setUsers={setUsers} />}
      {view === 'directory' && <Directory users={users} userProfile={userProfile}
onOpenDm={openDm} />}
      {view === 'chat' && <PublicChat userProfile={userProfile} messages={messages}
setMessages={setMessages} />}
      {view === 'admin-chat' && userProfile.isAdmin && <AdminChat
userProfile={userProfile} messages={adminMessages} setAdminMessages={setAdminMessages} />}
      {view === 'dm' && activeDmUser && <DirectChat userProfile={userProfile}
targetUser={activeDmUser} dms={dms} setDms={setDms} />}
      {view === 'leaderboard' && userProfile.isAdmin && <Leaderboard users={users} />}
      {view === 'profile' && <UserProfile userProfile={userProfile}
setUserProfile={setUserProfile} setUsers={setUsers} />}
      {view === 'ai' && <AICoach userProfile={userProfile} />}
    </main>
  </div>
</div>
);
}

// --- SUB-COMPONENTS ---

function NavItem({ icon, label, isActive, onClick, highlight }) {
  return (
    <button
      onClick={onClick}
      className={`w-full flex items-center gap-3 px-3 py-2.5 rounded-lg text-sm font-medium
transition-all duration-200 ${
        isActive
          ? (highlight ? 'bg-red-900/20 text-red-500' : 'bg-[#EAB308]/10 text-[#EAB308]')
          : 'text-gray-400 hover:bg-[#2A2E35] hover:text-white'
      }`}
    >
      {icon}
      {label}
    </button>
  );
}

// 1. Authentication Screen
function AuthScreen({ onComplete }) {
  const [step, setStep] = useState(1);
  const [formData, setFormData] = useState({ email: '', name: '', role: 'Setter', customRole:
'', masterKey: '' });
  const [error, setError] = useState('');

```

```

const isIsaac = formData.email.toLowerCase().trim() === 'isaacrosanazala@gmail.com';

const handleNext = (e) => {
  e.preventDefault();
  setError('');
  if (!formData.email || !formData.name) {
    setError("Please fill out all required fields.");
    return;
  }
  setStep(2);
};

const handleSubmit = (e) => {
  e.preventDefault();
  setError('');

  const isAdminRole = ROLES.ADMIN.includes(formData.role);

  // Bypass Master Key if email is Isaac's
  if (isAdminRole && !isIsaac && formData.masterKey !== MASTER_KEY) {
    setError("Invalid Master Key for HQ Leadership role.");
    return;
  }

  const finalRole = formData.role.startsWith('Other') ? formData.customRole :
formData.role;

  if (formData.role.startsWith('Other') && (!formData.customRole || !
formData.customRole.trim())) {
    setError("Please type your personalized role.");
    return;
  }

  onComplete({
    ...formData,
    role: finalRole,
    isAdminRole: isAdminRole
  });
};

return (
  <div className="min-h-screen flex items-center justify-center bg-[#111317] p-4">
    <div className="max-w-md w-full bg-[#15181C] rounded-xl border border-[#2A2E35]
shadow-2xl overflow-hidden">
      <div className="p-8 text-center border-b border-[#2A2E35] bg-[#1A1D23]">
        <SpartanHelmet className="w-14 h-14 text-[#EAB308] mx-auto mb-4 drop-shadow-md" />
        <h2 className="text-xl font-bold text-white uppercase tracking-widest">JK&R
Portal</h2>
        <p className="text-sm text-gray-400 mt-2">Sign in to the JK&R support PORTAL.</p>
      </div>

      <form onSubmit={step === 1 ? handleNext : handleSubmit} className="p-8 space-y-5">
        {error && (
          <div className="p-3 bg-red-900/30 border border-red-500/50 rounded text-red-200
text-sm text-center">
            {error}
          </div>
        )}
      </form>
    </div>
  )
);

```

```

{step === 1 ? (
  <div className="animate-fade-in space-y-5">
    <div>
      <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Work Email</label>
      <input
        type="email"
        className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4 py-3
text-white focus:outline-none focus:border-[#EAB308] transition-colors"
        placeholder="you@jkrconstruction.com"
        value={formData.email}
        onChange={(e) => setFormData({...formData, email: e.target.value})}
      />
    </div>

    <div>
      <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Full Name</label>
      <input
        type="text"
        className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4 py-3
text-white focus:outline-none focus:border-[#EAB308] transition-colors"
        placeholder="e.g., John Doe"
        value={formData.name}
        onChange={(e) => setFormData({...formData, name: e.target.value})}
      />
    </div>

    <button
      type="submit"
      className="w-full bg-[#EAB308] text-black font-bold uppercase tracking-wider
py-3.5 rounded-lg hover:bg-yellow-400 transition-colors mt-4 flex items-center justify-center
gap-2"
    >
      Continue <ChevronRight size={18} />
    </button>
  </div>
) : (
  <div className="animate-fade-in space-y-5">
    <div className="flex items-center gap-3 mb-6 bg-[#111317] p-3 rounded-lg border
border-[#2A2E35]">
      <div className="w-8 h-8 rounded-full bg-[#2A2E35] flex items-center justify-
center text-[#EAB308] font-bold">
        {formData.name.charAt(0).toUpperCase()}
      </div>
      <div>
        <p className="text-sm text-white font-medium">{formData.name}</p>
        <p className="text-xs text-gray-500">{formData.email}</p>
      </div>
      <button type="button" onClick={() => setStep(1)} className="ml-auto text-xs
text-[#EAB308] hover:underline">Edit</button>
    </div>

    <div>
      <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Select Role</label>
      <select

```

```

        className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4 py-3
text-white focus:outline-none focus:border-[#EAB308] transition-colors appearance-none"
        value={formData.role}
        onChange={(e) => setFormData({...formData, role: e.target.value})}
      >
        <optgroup label="Field Team">
          {ROLES.USER.map(r => <option key={r} value={r}>{r}</option>)}
        </optgroup>
        <optgroup label="HQ / Leadership">
          {ROLES.ADMIN.map(r => <option key={r} value={r}>{r}</option>)}
        </optgroup>
      </select>
    </div>

    {formData.role.startsWith('Other') && (
      <div className="animate-fade-in-down">
        <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Specify Role</label>
        <input
          type="text"
          className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4
py-3 text-white focus:outline-none focus:border-[#EAB308] transition-colors"
          placeholder="Type your personalized role..."
          value={formData.customRole}
          onChange={(e) => setFormData({...formData, customRole: e.target.value})}
        />
      </div>
    )}

    {ROLES.ADMIN.includes(formData.role) && !isIsaac && (
      <div className="animate-fade-in-down">
        <label className="flex items-center gap-2 text-xs font-semibold text-
[#EAB308] uppercase tracking-wider mb-2">
          <Lock size={14} /> Master Key Required
        </label>
        <input
          type="password"
          className="w-full bg-[#111317] border border-[#EAB308]/50 rounded-lg px-4
py-3 text-white focus:outline-none focus:border-[#EAB308] transition-colors"
          placeholder="Enter HQ Master Key"
          value={formData.masterKey}
          onChange={(e) => setFormData({...formData, masterKey: e.target.value})}
        />
      </div>
    )}

    {isIsaac && ROLES.ADMIN.includes(formData.role) && (
      <div className="text-xs text-green-500 flex items-center gap-2 bg-
green-900/20 p-2 rounded border border-green-800/50">
        <CheckCircle size={14} /> Master Key bypassed for authorized email.
      </div>
    )}

    <button
      type="submit"
      className="w-full bg-[#EAB308] text-black font-bold uppercase tracking-wider
py-3.5 rounded-lg hover:bg-yellow-400 transition-colors mt-4"
    >

```

```

        Enter Portal
      </button>
    </div>
  )}
</form>
</div>
</div>
);
}

// 2. Support System (Roadblocks & Q&A)
function SupportSystem({ userProfile, setUserProfile, tickets, setTickets, setUsers }) {
  const [newTicket, setNewTicket] = useState('');
  const [isAnonymous, setIsAnonymous] = useState(false);
  const [replyText, setReplyText] = useState({});

  const handleSubmitTicket = (e) => {
    e.preventDefault();
    if (!newTicket.trim()) return;

    const ticketData = {
      id: 'ticket-' + Date.now(),
      text: newTicket,
      authorId: userProfile.id,
      authorName: isAnonymous ? 'Anonymous Spartan' : userProfile.name,
      authorRole: isAnonymous ? 'Field Agent' : userProfile.role,
      isAnonymous,
      timestamp: Date.now(),
      status: 'open',
      replies: []
    };

    setTickets(prev => [ticketData, ...prev]);
    setNewTicket('');
    setIsAnonymous(false);
  };

  const handleReply = (ticketId) => {
    const text = replyText[ticketId];
    if (!text || !text.trim()) return;

    const newReply = {
      text,
      authorId: userProfile.id,
      authorName: userProfile.name,
      authorRole: userProfile.role,
      timestamp: Date.now()
    };

    setTickets(prev => prev.map(t => {
      if (t.id === ticketId) {
        return { ...t, replies: [...(t.replies || []), newReply], status: 'answered' };
      }
      return t;
    }));
  };

  if (userProfile.isAdmin) {
    setUsers(prev => prev.map(u => {

```



```

        if (u.id === userProfile.id) {
            return { ...u, score: (u.score || 0) + 1 };
        }
        return u;
    }));
    setUserProfile(prev => ({ ...prev, score: (prev.score || 0) + 1 }));
}

setReplyText(prev => ({ ...prev, [ticketId]: '' }));
};

return (
    <div className="max-w-4xl mx-auto space-y-8">
        <div className="bg-[#15181C] border border-[#2A2E35] rounded-xl p-6 shadow-lg">
            <h3 className="text-sm font-bold text-gray-400 uppercase tracking-widest mb-4">Report
a Roadblock or Ask HQ</h3>
            <form onSubmit={handleSubmitTicket}>
                <textarea
                    className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg p-4 text-white
focus:outline-none focus:border-[#EAB308] resize-none transition-colors"
                    rows="3"
                    placeholder="What objection are you struggling with? What do you need help on?"
                    value={newTicket}
                    onChange={(e) => setNewTicket(e.target.value)}
                ></textarea>
                <div className="flex items-center justify-between mt-4">
                    <label className="flex items-center gap-2 cursor-pointer group">
                        <div className={`w-5 h-5 rounded flex items-center justify-center border
transition-colors ${isAnonymous ? 'bg-[#EAB308] border-[#EAB308]' : 'border-[#2A2E35] bg-
[#111317]'}`}>
                            {isAnonymous && <CheckCircle size={14} className="text-black" />}
                        </div>
                        <input type="checkbox" className="hidden" checked={isAnonymous} onChange={(e)
=> setIsAnonymous(e.target.checked)} />
                        <span className="text-sm text-gray-400 group-hover:text-white transition-colors
flex items-center gap-1">
                            <EyeOff size={14} /> Post Anonymously
                        </span>
                    </label>
                    <button
                        type="submit"
                        disabled={!newTicket.trim()}
                        className="bg-[#EAB308] text-black font-bold px-6 py-2 rounded-lg flex items-
center gap-2 hover:bg-yellow-400 disabled:opacity-50 disabled:cursor-not-allowed transition-
colors"
                    >
                        Submit <Send size={16} />
                    </button>
                </div>
            </form>
        </div>

        <div className="space-y-4">
            <h3 className="text-xs font-bold text-[#EAB308] uppercase tracking-widest
mb-6">Active Discussions</h3>
            {tickets.length === 0 ? (
                <div className="text-center py-12 text-gray-500 border border-dashed border-
[#2A2E35] rounded-xl">

```

```

        No active roadblocks reported. You guys are crushing it!
    </div>
  ) : (
    tickets.map(ticket => (
      <div key={ticket.id} className="bg-[#15181C] border border-[#2A2E35] rounded-xl
overflow-hidden transition-all hover:border-gray-600">
        <div className="p-6">
          <div className="flex items-start justify-between mb-3">
            <div className="flex items-center gap-3">
              <div className={`w-10 h-10 rounded-full flex items-center justify-center
font-bold ${ticket.isAnonymous ? 'bg-gray-800 text-gray-500' : 'bg-[#2A2E35] text-[#EAB308]'}
`}>
                {ticket.isAnonymous ? '?' : ticket.authorName.charAt(0).toUpperCase()}
              </div>
              <div>
                <h4 className="text-white font-medium flex items-center gap-2">
                  {ticket.authorName}
                  {ticket.isAnonymous && <span className="bg-gray-800 text-xs px-2
py-0.5 rounded text-gray-400">Anonymous</span>}
                </h4>
                <p className="text-xs text-gray-500">{ticket.authorRole} • {new
Date(ticket.timestamp).toLocaleDateString()} {new
Date(ticket.timestamp).toLocaleTimeString([], {hour: '2-digit', minute: '2-digit'})}</p>
              </div>
            </div>
            <div className={`px-3 py-1 rounded-full text-xs font-bold uppercase
tracking-wider ${ticket.status === 'answered' ? 'bg-green-900/30 text-green-400 border
border-green-800' : 'bg-yellow-900/30 text-yellow-500 border border-yellow-800'}`}>
              {ticket.status}
            </div>
          </div>
          <p className="text-gray-300 mt-4 leading-relaxed">{ticket.text}</p>
        </div>

        {ticket.replies && ticket.replies.length > 0 && (
          <div className="bg-[#1A1D23] border-t border-[#2A2E35] p-6 space-y-4">
            {ticket.replies.map((reply, idx) => (
              <div key={idx} className="flex items-start gap-3 pl-4 border-l-2 border-
[#EAB308]">
                <div className="w-8 h-8 rounded-full bg-[#2A2E35] flex items-center
justify-center text-white text-xs font-bold shrink-0">
                  {reply.authorName.charAt(0)}
                </div>
                <div>
                  <div className="flex items-baseline gap-2">
                    <span className="text-white font-medium text-sm">{reply.authorName}
</span>
                    <span className="text-xs text-[#EAB308]">{reply.authorRole}</span>
                    <span className="text-xs text-gray-500">• {new
Date(reply.timestamp).toLocaleTimeString([], {hour: '2-digit', minute: '2-digit'})}</span>
                  </div>
                  <p className="text-gray-400 text-sm mt-1">{reply.text}</p>
                </div>
              </div>
            )})}
          </div>
        )}
      </div>
    )}
  )}

```

```

        <div className="p-4 border-t border-[#2A2E35] bg-[#111317] flex gap-3">
          <input
            type="text"
            placeholder={userProfile.isAdmin ? "Provide an HQ response..." : "Add to
the discussion..."}
            className="flex-1 bg-transparent border border-[#2A2E35] rounded-lg px-4
text-sm text-white focus:outline-none focus:border-[#EAB308] transition-colors"
            value={replyText[ticket.id] || ''}
            onChange={(e) => setReplyText({ ...replyText, [ticket.id]:
e.target.value })}
            onKeyDown={(e) => e.key === 'Enter' && handleReply(ticket.id)}
          />
          <button
            onClick={() => handleReply(ticket.id)}
            className="bg-[#2A2E35] text-white px-4 rounded-lg hover:bg-[#EAB308]
hover:text-black transition-colors flex items-center justify-center"
          >
            <Send size={16} />
          </button>
        </div>
      </div>
    ))
  )}
</div>
</div>
);
}

// 3. Team Directory
function Directory({ users, userProfile, onOpenDm }) {
  // Logic: Admins see everyone. Setters/Closers only see non-admins.
  const visibleUsers = userProfile.isAdmin
    ? users
    : users.filter(u => !u.isAdmin);

  return (
    <div className="max-w-5xl mx-auto">
      <div className="mb-8">
        <h2 className="text-2xl font-bold text-white tracking-wide">Team Roster</h2>
        <p className="text-gray-400 mt-1">Connect with {userProfile.isAdmin ? 'everyone in
the organization' : 'your fellow field reps'}.</p>
      </div>

      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-4">
        {visibleUsers.map(user => {
          const isMe = user.id === userProfile.id;
          return (
            <div key={user.id} className="bg-[#15181C] border border-[#2A2E35] rounded-xl p-5
flex items-center justify-between group hover:border-gray-600 transition-colors">
              <div className="flex items-center gap-3">
                <div className={`w-12 h-12 rounded-full flex items-center justify-center
font-bold text-lg ${user.isAdmin ? 'bg-amber-900/30 text-[#EAB308] border border-[#EAB308]/
30' : 'bg-[#2A2E35] text-white'} `}>
                  {user.name.charAt(0).toUpperCase()}
                </div>
              </div>
              <div>
                <h3 className="text-white font-medium flex items-center gap-2">
                  {user.name} {isMe && <span className="text-[10px] bg-gray-800 text-

```

```

    gray-400 px-2 py-0.5 rounded">You</span>}
      </h3>
      <p className="text-xs text-gray-500">{user.role}</p>
    </div>
  </div>

  {!isMe && (
    <button
      onClick={() => onOpenDm(user)}
      className="w-10 h-10 rounded-full bg-[#111317] border border-[#2A2E35] flex
items-center justify-center text-gray-400 hover:text-[#EAB308] hover:border-[#EAB308]
transition-colors"
      title={`Message ${user.name}`}
    >
      <MessageCircle size={18} />
    </button>
  )}
</div>
);
  )}
</div>
</div>
);
}

// 4. Public Chat
function PublicChat({ userProfile, messages, setMessages }) {
  const [newMsg, setNewMsg] = useState('');
  const [isAnonymous, setIsAnonymous] = useState(false);
  const messagesEndRef = useRef(null);

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages]);

  const handleSend = (e) => {
    e.preventDefault();
    if (!newMsg.trim()) return;

    const messageData = {
      id: 'msg-' + Date.now(),
      text: newMsg,
      authorId: userProfile.id,
      authorName: isAnonymous ? 'Anonymous' : userProfile.name,
      authorRole: userProfile.role,
      isAdmin: userProfile.isAdmin && !isAnonymous,
      timestamp: Date.now()
    };

    setMessages(prev => [...prev, messageData]);
    setNewMsg('');
  };

  return (

```

```

<div className="max-w-4xl mx-auto h-[calc(100vh-8rem)] flex flex-col bg-[#15181C] border
border-[#2A2E35] rounded-xl overflow-hidden shadow-2xl">
  <div className="px-6 py-4 border-b border-[#2A2E35] bg-[#1A1D23] flex items-center
justify-between">
    <div>
      <h3 className="text-white font-bold tracking-wide">Company Comm-Link</h3>
      <p className="text-xs text-gray-500">Public channel for all Setters, Closers, and
HQ.</p>
    </div>
    <div className="flex items-center gap-2">
      <span className="w-2 h-2 rounded-full bg-green-500 animate-pulse"></span>
      <span className="text-xs text-gray-400 uppercase tracking-widest font-
semibold">Live</span>
    </div>
  </div>

  <div className="flex-1 overflow-y-auto p-6 space-y-6">
    {messages.length === 0 ? (
      <div className="h-full flex flex-col items-center justify-center text-gray-500">
        <MessageSquare size={48} className="mb-4 opacity-20" />
        <p>The comms channel is quiet. Be the first to speak.</p>
      </div>
    ) : (
      messages.map((msg) => {
        const isMe = msg.authorId === userProfile.id && !msg.isAnonymous;
        return (
          <div key={msg.id} className={`flex flex-col ${isMe ? 'items-end' : 'items-
start'}`>
            <div className="flex items-baseline gap-2 mb-1 px-1">
              <span className={`text-xs font-semibold ${msg.isAdmin ? 'text-[#EAB308]' :
'text-gray-400'}`>
                {msg.authorName} {msg.isAdmin && '🛡️'}
              </span>
              <span className="text-[10px] text-gray-600">
                {new Date(msg.timestamp).toLocaleTimeString([], {hour: '2-digit',
minute: '2-digit'})}
              </span>
            </div>
            <div className={`px-4 py-2.5 rounded-2xl max-w-[80%] text-sm leading-relaxed
${
              isMe
                ? 'bg-[#EAB308] text-black rounded-tr-sm'
                : 'bg-[#2A2E35] text-gray-200 rounded-tl-sm border border-gray-700/50'
            }`>
              {msg.text}
            </div>
          </div>
        );
      })
    )}
    <div ref={messagesEndRef} />
  </div>

  <div className="p-4 bg-[#1A1D23] border-t border-[#2A2E35]">
    <form onSubmit={handleSend} className="flex gap-3 items-center">
      <label className="flex items-center gap-1 cursor-pointer group" title="Toggle
Anonymous Mode">
        <div className={`w-8 h-8 rounded-full flex items-center justify-center

```

```

transition-colors ${isAnonymous ? 'bg-gray-700 text-white' : 'bg-[#111317] border border-
[#2A2E35] text-gray-500 group-hover:text-gray-300'}``>
    <EyeOff size={16} />
  </div>
  <input type="checkbox" className="hidden" checked={isAnonymous} onChange={(e) =>
setIsAnonymous(e.target.checked)} />
</label>
<input
  type="text"
  className="flex-1 bg-[#111317] border border-[#2A2E35] rounded-full px-5 py-3
text-sm text-white focus:outline-none focus:border-[#EAB308] transition-colors"
  placeholder={isAnonymous ? "Typing anonymously..." : "Transmit message to HQ and
Field..."}
  value={newMsg}
  onChange={(e) => setNewMsg(e.target.value)}
/>
<button
  type="submit"
  disabled={!newMsg.trim()}
  className="w-12 h-12 rounded-full bg-[#EAB308] text-black flex items-center
justify-center hover:bg-yellow-400 disabled:opacity-50 disabled:cursor-not-allowed
transition-colors shrink-0"
  >
  <Send size={18} className="ml-1" />
</button>
</form>
</div>
</div>
);
}

```

// 5. Admin Private Chat

```

function AdminChat({ userProfile, messages, setAdminMessages }) {
  const [newMsg, setNewMsg] = useState('');
  const messagesEndRef = useRef(null);

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  };

  useEffect(() => {
    scrollToBottom();
  }, [messages]);

  const handleSend = (e) => {
    e.preventDefault();
    if (!newMsg.trim()) return;

    const messageData = {
      id: 'amsg-' + Date.now(),
      text: newMsg,
      authorId: userProfile.id,
      authorName: userProfile.name,
      authorRole: userProfile.role,
      timestamp: Date.now()
    };

    setAdminMessages(prev => [...prev, messageData]);
  };
}

```

```

    setNewMsg('');
  };

  return (
    <div className="max-w-4xl mx-auto h-[calc(100vh-8rem)] flex flex-col bg-[#15181C] border
border-red-900/50 rounded-xl overflow-hidden shadow-2xl">
      <div className="px-6 py-4 border-b border-red-900/50 bg-[#1A1D23] flex items-center
justify-between">
        <div>
          <h3 className="text-red-500 font-bold tracking-wide flex items-center
gap-2"><ShieldAlert size={18} /> HQ Secure Comms</h3>
          <p className="text-xs text-red-900/80">Private channel visible only to
Leadership.</p>
        </div>
      </div>

      <div className="flex-1 overflow-y-auto p-6 space-y-6">
        {messages.length === 0 ? (
          <div className="h-full flex flex-col items-center justify-center text-gray-500">
            <ShieldAlert size={48} className="mb-4 opacity-20 text-red-500" />
            <p>Admin channel initialized. Secure communications only.</p>
          </div>
        ) : (
          messages.map((msg) => {
            const isMe = msg.authorId === userProfile.id;
            return (
              <div key={msg.id} className={`flex flex-col ${isMe ? 'items-end' : 'items-
start'}}`>
                <div className="flex items-baseline gap-2 mb-1 px-1">
                  <span className="text-xs font-semibold text-red-400">
                    {msg.authorName}
                  </span>
                  <span className="text-[10px] text-gray-600">
                    {new Date(msg.timestamp).toLocaleTimeString([], {hour: '2-digit',
minute: '2-digit'})}
                  </span>
                </div>
                <div className={`px-4 py-2.5 rounded-2xl max-w-[80%] text-sm leading-relaxed
${
                    isMe
                      ? 'bg-red-900/40 text-red-100 border border-red-800/50 rounded-tr-sm'
                      : 'bg-[#2A2E35] text-gray-200 rounded-tl-sm border border-gray-700/50'
                  }`>
                  {msg.text}
                </div>
              </div>
            );
          })
        )}
        <div ref={messagesEndRef} />
      </div>

      <div className="p-4 bg-[#1A1D23] border-t border-red-900/50">
        <form onSubmit={handleSend} className="flex gap-3 items-center">
          <input
            type="text"
            className="flex-1 bg-[#111317] border border-[#2A2E35] rounded-full px-5 py-3
text-sm text-white focus:outline-none focus:border-red-500 transition-colors"

```

```

        placeholder="Transmit secure message to HQ..."
        value={newMsg}
        onChange={(e) => setNewMsg(e.target.value)}
      />
      <button
        type="submit"
        disabled={!newMsg.trim()}
        className="w-12 h-12 rounded-full bg-red-900/80 text-white flex items-center
justify-center hover:bg-red-700 disabled:opacity-50 disabled:cursor-not-allowed transition-
colors shrink-0"
      >
        <Send size={18} className="ml-1" />
      </button>
    </form>
  </div>
</div>
);
}

// 6. Direct Messaging (1-on-1)
function DirectChat({ userProfile, targetUser, dms, setDms }) {
  const [newMsg, setNewMsg] = useState('');
  const messagesEndRef = useRef(null);

  // Filter messages for this specific conversation
  const conversation = dms.filter(m =>
    (m.senderId === userProfile.id && m.receiverId === targetUser.id) ||
    (m.senderId === targetUser.id && m.receiverId === userProfile.id)
  );

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  };

  useEffect(() => {
    scrollToBottom();
  }, [conversation]);

  const handleSend = (e) => {
    e.preventDefault();
    if (!newMsg.trim()) return;

    const messageData = {
      id: 'dm-' + Date.now(),
      text: newMsg,
      senderId: userProfile.id,
      receiverId: targetUser.id,
      timestamp: Date.now()
    };

    setDms(prev => [...prev, messageData]);
    setNewMsg('');
  };

  return (
    <div className="max-w-4xl mx-auto h-[calc(100vh-8rem)] flex flex-col bg-[#15181C] border
border-[#2A2E35] rounded-xl overflow-hidden shadow-2xl">
      <div className="px-6 py-4 border-b border-[#2A2E35] bg-[#1A1D23] flex items-center

```



```

gap-4">
  <div className="w-10 h-10 rounded-full bg-[#2A2E35] flex items-center justify-center
font-bold text-white">
    {targetUser.name.charAt(0).toUpperCase()}
  </div>
  <div>
    <h3 className="text-white font-bold tracking-wide">{targetUser.name}</h3>
    <p className="text-xs text-[#EAB308]">{targetUser.role}</p>
  </div>
</div>

<div className="flex-1 overflow-y-auto p-6 space-y-6">
  {conversation.length === 0 ? (
    <div className="h-full flex flex-col items-center justify-center text-gray-500">
      <MessageCircle size={48} className="mb-4 opacity-20" />
      <p>No messages yet. Start the conversation with {targetUser.name.split(' ')[0]}</p>
    </div>
  ) : (
    conversation.map((msg) => {
      const isMe = msg.senderId === userProfile.id;
      return (
        <div key={msg.id} className={`flex flex-col ${isMe ? 'items-end' : 'items-start'} `>
          <div className="flex items-baseline gap-2 mb-1 px-1">
            <span className="text-[10px] text-gray-600">
              {new Date(msg.timestamp).toLocaleTimeString([], {hour: '2-digit',
minute: '2-digit'})}
            </span>
          </div>
          <div className={`px-4 py-2.5 rounded-2xl max-w-[80%] text-sm leading-relaxed
${
      isMe
        ? 'bg-[#EAB308] text-black rounded-tr-sm'
        : 'bg-[#2A2E35] text-gray-200 rounded-tl-sm border border-gray-700/50'
    } `>
            {msg.text}
          </div>
        </div>
      );
    })
  )}
  <div ref={messagesEndRef} />
</div>

<div className="p-4 bg-[#1A1D23] border-t border-[#2A2E35]">
  <form onSubmit={handleSend} className="flex gap-3 items-center">
    <input
      type="text"
      className="flex-1 bg-[#111317] border border-[#2A2E35] rounded-full px-5 py-3
text-sm text-white focus:outline-none focus:border-[#EAB308] transition-colors"
      placeholder={`Message ${targetUser.name.split(' ')[0]}...`}
      value={newMsg}
      onChange={(e) => setNewMsg(e.target.value)}
    />
    <button
      type="submit"
      disabled={!newMsg.trim()}

```

```

        className="w-12 h-12 rounded-full bg-[#EAB308] text-black flex items-center
justify-center hover:bg-yellow-400 disabled:opacity-50 disabled:cursor-not-allowed
transition-colors shrink-0"
      >
        <Send size={18} className="ml-1" />
      </button>
    </form>
  </div>
</div>
);
}

// 7. Spartan AI Coach (Gemini API Integration)
function AICoach({ userProfile }) {
  const [chatHistory, setChatHistory] = useState([
    { role: 'model', text: `Welcome to the Spartan AI Coach, ${userProfile.name.split(' ')}
[0]}. I'm here to help you crush objections, frame pitches, and provide neuro-linguistic
advice on the fly. What's the toughest objection you're facing today?` }
  ]);
  const [inputText, setInputText] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const messagesEndRef = useRef(null);

  const scrollToBottom = () => {
    messagesEndRef.current?.scrollIntoView({ behavior: "smooth" });
  };

  useEffect(() => {
    scrollToBottom();
  }, [chatHistory]);

  const handleSend = async (e) => {
    e.preventDefault();
    if (!inputText.trim()) return;

    const newHistory = [...chatHistory, { role: 'user', text: inputText }];
    setChatHistory(newHistory);
    setInputText('');
    setIsLoading(true);

    try {
      const response = await fetch(`https://generativelanguage.googleapis.com/v1beta/models/
gemini-2.5-flash-preview-09-2025:generateContent?key=${apiKey}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          contents: newHistory.map(m => ({ role: m.role, parts: [{ text: m.text }] })),
          systemInstruction: {
            parts: [{
              text: "You are an expert sales coach for JK&R Construction, specializing in
Impact Windows & Doors. Give actionable, concise advice to door-to-door setters and closers.
Focus on humanizing questions, neuroscience, handling resistance, and maintaining a 'Spartan'
mindset. Keep responses under 3 paragraphs and highly practical."
            }]
          }
        })
      });
    }
  });
}

```

```

    const data = await response.json();
    const reply = data.candidates?.[0]?.content?.parts?.[0]?.text || "Communication with HQ
AI interrupted. Please try again.";

    setChatHistory([...newHistory, { role: 'model', text: reply }]);
  } catch (err) {
    console.error("AI Error:", err);
    setChatHistory([...newHistory, { role: 'model', text: "Error: Neural link offline.
Ensure API key is configured in the environment." }]);
  } finally {
    setIsLoading(false);
  }
};

return (
  <div className="max-w-4xl mx-auto h-[calc(100vh-8rem)] flex flex-col bg-[#15181C] border
border-blue-900/30 rounded-xl overflow-hidden shadow-2xl">
    <div className="px-6 py-4 border-b border-blue-900/30 bg-[#1A1D23] flex items-center
justify-between">
      <div className="flex items-center gap-3">
        <Bot className="text-blue-400" size={24} />
        <div>
          <h3 className="text-blue-400 font-bold tracking-wide">Spartan AI Sales Coach</h3>
          <p className="text-xs text-blue-900/80">Neuroscience-driven LLM trained for JK&R
field support.</p>
        </div>
      </div>
    </div>

    <div className="flex-1 overflow-y-auto p-6 space-y-6 bg-gradient-to-b from-[#15181C]
to-[#111317]">
      {chatHistory.map((msg, idx) => {
        const isUser = msg.role === 'user';
        return (
          <div key={idx} className={`flex flex-col ${isUser ? 'items-end' : 'items-start'}
`>
            <div className="flex items-baseline gap-2 mb-1 px-1">
              <span className={`text-xs font-semibold ${isUser ? 'text-gray-400' : 'text-
blue-400'}`>
                {isUser ? userProfile.name : 'Spartan AI'}
              </span>
            </div>
            <div className={`px-4 py-3 rounded-2xl max-w-[85%] text-sm leading-relaxed
whitespace-pre-wrap ${
              isUser
                ? 'bg-[#2A2E35] text-white rounded-tr-sm border border-gray-700/50'
                : 'bg-blue-900/10 text-blue-100 rounded-tl-sm border border-blue-800/30'
            }`>
              {msg.text}
            </div>
          </div>
        );
      })}
      {isLoading && (
        <div className="flex items-start">
          <div className="px-4 py-3 rounded-2xl bg-blue-900/10 text-blue-400 rounded-tl-sm
border border-blue-800/30 text-sm flex items-center gap-2">
            <div className="w-2 h-2 bg-blue-400 rounded-full animate-bounce"></div>

```

```

        <div className="w-2 h-2 bg-blue-400 rounded-full animate-bounce"
style={{animationDelay: '0.2s'}}></div>
        <div className="w-2 h-2 bg-blue-400 rounded-full animate-bounce"
style={{animationDelay: '0.4s'}}></div>
    </div>
</div>
)}
<div ref={messagesEndRef} />
</div>

<div className="p-4 bg-[#1A1D23] border-t border-blue-900/30">
    <form onSubmit={handleSend} className="flex gap-3 items-center">
        <input
            type="text"
            className="flex-1 bg-[#111317] border border-[#2A2E35] rounded-full px-5 py-3
text-sm text-white focus:outline-none focus:border-blue-500 transition-colors"
            placeholder="Ask the AI coach for advice..."
            value={inputText}
            onChange={(e) => setInputText(e.target.value)}
            disabled={isLoading}
        />
        <button
            type="submit"
            disabled={!inputText.trim() || isLoading}
            className="w-12 h-12 rounded-full bg-blue-600 text-white flex items-center
justify-center hover:bg-blue-500 disabled:opacity-50 disabled:cursor-not-allowed transition-
colors shrink-0"
        >
            <Send size={18} className="ml-1" />
        </button>
    </form>
</div>
</div>
);
}

// 8. Leaderboard
function Leaderboard({ users }) {
    const admins = users
        .filter(u => u.isAdmin)
        .sort((a, b) => (b.score || 0) - (a.score || 0));

    return (
        <div className="max-w-3xl mx-auto">
            <div className="mb-8 text-center">
                <Trophy size={48} className="text-[#EAB308] mx-auto mb-4" />
                <h2 className="text-2xl font-bold text-white uppercase tracking-widest">HQ Engagement
Roster</h2>
                <p className="text-gray-400 mt-2">Tracking leadership responsiveness to field
roadblocks.</p>
            </div>

            <div className="space-y-3">
                {admins.map((admin, idx) => (
                    <div key={admin.id} className="bg-[#15181C] border border-[#2A2E35] rounded-xl p-4
flex items-center gap-6 relative overflow-hidden group hover:border-[#EAB308]/50 transition-
colors">
                        <div className={`w-12 text-center text-2xl font-black italic ${idx === 0 ? 'text-

```

```

[#EAB308]' : idx === 1 ? 'text-gray-300' : idx === 2 ? 'text-amber-700' : 'text-gray-600'}}`>
    #{idx + 1}
  </div>

  <div className="flex-1">
    <h3 className="text-lg font-bold text-white">{admin.name}</h3>
    <p className="text-sm text-[#EAB308]">{admin.role}</p>
  </div>

  <div className="text-right pr-6">
    <div className="text-3xl font-black text-white">{admin.score || 0}</div>
    <div className="text-[10px] text-gray-500 uppercase tracking-widest font-
bold">Answers Provided</div>
  </div>

  {idx === 0 && (
    <div className="absolute right-0 top-0 bottom-0 w-32 bg-gradient-to-l from-
[#EAB308]/10 to-transparent pointer-events-none" />
  )}
</div>
  )}
  {admins.length === 0 && (
    <div className="text-center py-12 text-gray-500">No HQ data available yet.</div>
  )}
</div>
</div>
);
}

// 9. User Profile
function UserProfile({ userProfile, setUserProfile, setUsers }) {
  const [isEditing, setIsEditing] = useState(false);

  const isCurrentlyAdmin = userProfile.isAdmin;
  const standardRoles = isCurrentlyAdmin ? ROLES.ADMIN : ROLES.USER;
  const isCustomRole = !standardRoles.includes(userProfile.role);

  const initialRoleSelect = isCustomRole ? (isCurrentlyAdmin ? 'Other (HQ)' : 'Other
(Field)') : userProfile.role;
  const initialCustomRole = isCustomRole ? userProfile.role : '';

  const [formData, setFormData] = useState({
    name: userProfile.name,
    roleSelect: initialRoleSelect,
    customRole: initialCustomRole
  });

  const handleSave = () => {
    const finalRole = formData.roleSelect.startsWith('Other') ? formData.customRole :
formData.roleSelect;

    if (formData.roleSelect.startsWith('Other') && !formData.customRole.trim()) {
      alert("Please specify your custom role.");
      return;
    }

    setUsers(prev => prev.map(u => u.id === userProfile.id ? { ...u, name: formData.name,
role: finalRole } : u));
  }
}

```

```

    setUserProfile(prev => ({ ...prev, name: formData.name, role: finalRole }));
    setIsEditing(false);
  };

  return (
    <div className="max-w-2xl mx-auto">
      <div className="bg-[#15181C] border border-[#2A2E35] rounded-xl overflow-hidden shadow-2xl">
        <div className="h-32 bg-[#1A1D23] border-b border-[#2A2E35] relative">
          <div className="absolute -bottom-10 left-8 w-24 h-24 rounded-xl bg-[#111317] border-4 border-[#15181C] flex items-center justify-center text-4xl font-bold text-[#EAB308] shadow-lg">
            {userProfile.name.charAt(0).toUpperCase()}
          </div>
        </div>

        <div className="pt-14 px-8 pb-8">
          <div className="flex justify-between items-start mb-8">
            <div>
              <h2 className="text-2xl font-bold text-white">{userProfile.name}</h2>
              <p className="text-[#EAB308] font-medium mt-1">{userProfile.role}
                {userProfile.isAdmin && ' • HQ Level'}</p>
              <p className="text-xs text-gray-500 mt-1">Joined {new
                Date(userProfile.joinedAt).toLocaleDateString()}</p>
            </div>
            {!isEditing ? (
              <button
                onClick={() => setIsEditing(true)}
                className="px-4 py-2 bg-[#2A2E35] text-white text-sm rounded hover:bg-gray-700 transition-colors"
                >
                  Edit Profile
                </button>
            ) : (
              <div className="flex gap-2">
                <button
                  onClick={() => setIsEditing(false)}
                  className="px-4 py-2 border border-[#2A2E35] text-gray-400 text-sm rounded hover:text-white transition-colors"
                  >
                    Cancel
                  </button>
                <button
                  onClick={handleSave}
                  className="px-4 py-2 bg-[#EAB308] text-black font-bold text-sm rounded hover:bg-yellow-400 transition-colors"
                  >
                    Save
                  </button>
                </div>
            )}
          </div>

          {isEditing && (
            <div className="space-y-4 border-t border-[#2A2E35] pt-6">
              <div>
                <label className="block text-xs font-semibold text-gray-400 uppercase tracking-wider mb-2">Display Name</label>

```

```

        <input
          type="text"
          className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4
py-3 text-white focus:outline-none focus:border-[#EAB308]"
          value={formData.name}
          onChange={(e) => setFormData({...formData, name: e.target.value})}
        />
      </div>
      {!userProfile.isAdmin && (
        <div>
          <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Role</label>
          <select
            className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4
py-3 text-white focus:outline-none focus:border-[#EAB308] appearance-none"
            value={formData.roleSelect}
            onChange={(e) => setFormData({...formData, roleSelect:
e.target.value})}
          >
            {ROLES.USER.map(r => <option key={r} value={r}>{r}</option>)}
          </select>
        </div>
      )}
      {userProfile.isAdmin && (
        <div>
          <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Role</label>
          <select
            className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4
py-3 text-white focus:outline-none focus:border-[#EAB308] appearance-none"
            value={formData.roleSelect}
            onChange={(e) => setFormData({...formData, roleSelect:
e.target.value})}
          >
            {ROLES.ADMIN.map(r => <option key={r} value={r}>{r}</option>)}
          </select>
        </div>
      )}

      {formData.roleSelect.startsWith('Other') && (
        <div className="animate-fade-in-down">
          <label className="block text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2">Specify Role</label>
          <input
            type="text"
            className="w-full bg-[#111317] border border-[#2A2E35] rounded-lg px-4
py-3 text-white focus:outline-none focus:border-[#EAB308] transition-colors"
            placeholder="Type your personalized role..."
            value={formData.customRole}
            onChange={(e) => setFormData({...formData, customRole:
e.target.value})}
          />
        </div>
      )}
    </div>
  )}
</div>
</div>

```

```
</div>  
);  
}
```